Technische Universität Braunschweig

Chair for Chip Design for Embedded Computing

Hardware-Software-Systeme

Kapitel 03 - VHDL - Hardware-Beschreibungssprache

Prof. Dr.-Ing. Guillermo Payá Vayá, 4. November 2025

Standard-Operatoren

Operatoren: Einleitung

- Operatoren dienen dazu, ein oder zwei Operanden zu transformieren
- Die Operanden eines Operators müssen vom selben Datentyp sein
  - Aber durch Überladen von Operatoren kann ein Operator auch auf Operanden unterschiedlichen Datentyps arbeiten (`packages`)
- Es gibt folgende Klassen von Operatoren
  - Logische Operatoren
  - Relationale Operatoren
  - Schiebe-Operatoren
  - Sonstige Operatoren (Verknüpfung)
  - *Arithmetische Operatoren (Additive, Multiplikation/Division)*

Logische Operatoren

c => f 8 => 1111 1000

b => 1 f => 0000 0111

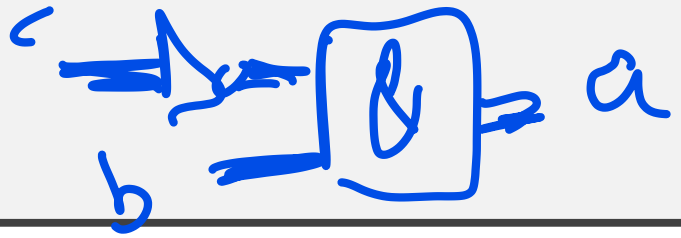
0000 0111

1111 1111

- Vorhandene Operatoren
  → `and/nand, or/nor, xor/xnor, not`
- Kompatible Datentypen
  - Standard VHDL: `bit, bit_vector, boolean`
  - std_logic_1164: `std_ulogic[_vector], std_logic[_vector]`
- Nutzungsregeln
  - Die Operanden und das Ergebnis müssen zum selben Datentyp gehören und ggf. dieselbe Breite haben
    Ausnahme: std_logic und std_ulogic (**Kein _vector !**)
  - Bei Vektoren wird die logische Operation bitweise angewandt

07

```
  signal a, b, c : std_ulogic_vector(7 downto 0);
  signal wr_en   : boolean;
begin
  a     <= b and not (c);
  wr_en <= true;
```



Logische Operatoren

Relationale Operatoren

- Vergleich von 2 Operanden desselben Typs. Ergebnis: `boolean`
- Vorhandene Operatoren
  - `gleich` `=`
  - `ungleich` `/=`
  - `kleiner` `<`
  - `größer` `>`
  - `größer gleich` `>=`
  - `kleiner gleich` `<=`

<=

- Kompatible Datentypen
  - Standard VHDL: `boolean`, `bit`, `character`, `integer`, `real`, `time`, `string` und `bit_vector`
  - numeric_std: `unsigned`, `signed`
    → unsigned & signed dürfen mit natural bzw. integer verglichen
- **Achtung:** Wenn Vektoren unterschiedlicher Länge verglichen werden, werden sie linksbündig verglichen.

  Bsp.: `(1011 < 110)` gibt ein `true` aus

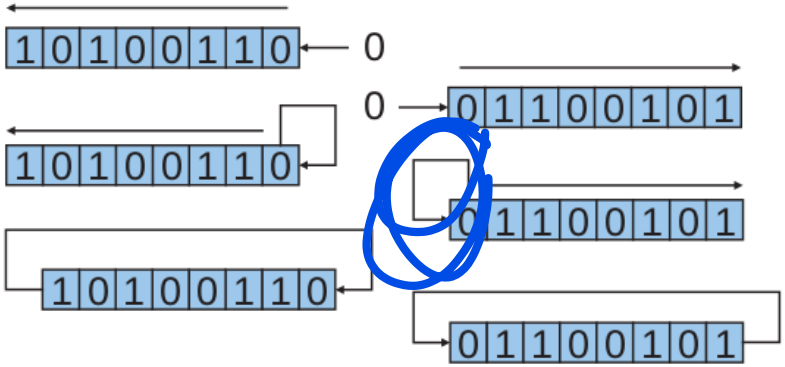
1 0 1 x
1 1 0

Rationale Operatoren

Schiebe-Operatoren

b <= a sll 3

- Vorhandene Operatoren
  - `sll` - shift left logical
  - `srl` - shift right logical
  - `sla` - shift left arithmetic
  - `sra` - shift right arithmetic
  - `rol` - rotate left logical
  - `ror` - rotate right logical



- Datentypen & Syntax (VHDL-Standard)

  `<ziel> <= <quelle> <operator> <Schiebpos>`
  - `<ziel>` und `<quelle>` müssen gleiche Typ und Breite haben
  - `<Schiebpos>` vom Typ `integer`
    (bei <0 umgekehrte Richtung)

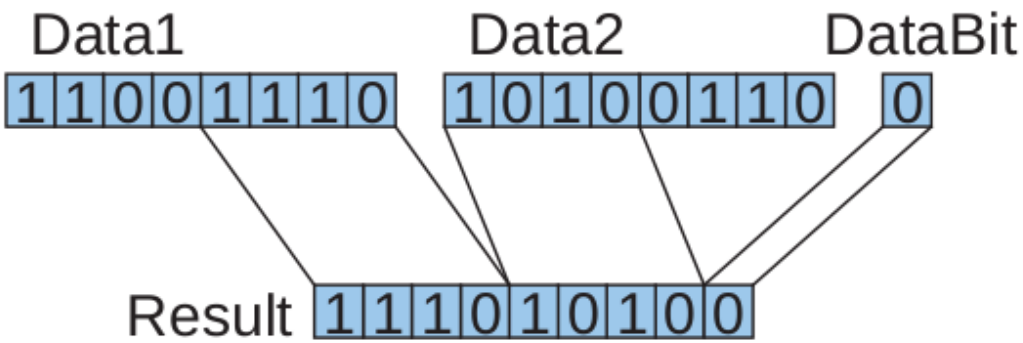
```
my_bitvec <= your_bitvec sra 3;
```

Schiebe Operatoren

Verknüpfungsoperator

- Mit dem Verknüpfungsoperator (&) können eindimensionale `arrays` und skalare Elemente desselben Datentyps zu einem `array` dieses Datentyps kombienert werden
- Die Länge des Zielobjekts muss der Summer der Längen aller Operanden entsprechen

```
  signal Data1, Data2 : std_ulogic_vector(7 downto 0);
  signal DataBit      : std_ulogic;
  signal Result       : std_ulogic_vector(8 downto 0);
begin
  Result <= (Data1(3 downto 0) & Data2(7 downto 4) & DataBit);
```



Verknüpfungs Op.

Operatoren: Präzedenz und Kombination

- Die zur Verfügung stehenden Operatoren dürfen kombiniert werden, vorausgesetzt alle Typ- und Breitevorgaben werden eingehalten

Operatorklasse						
Logisch	and	or	nand	nor	xor	xnor
Relational	=	/=	<	<=	>	>=
Schieben	sll	srl	sla	sra	rol	ror
Additiv	+	-	&			
Unär	+	-				
Multiplikativ	*	/	mod	rem		
Sonstiges	**	abs	not			

Priorität

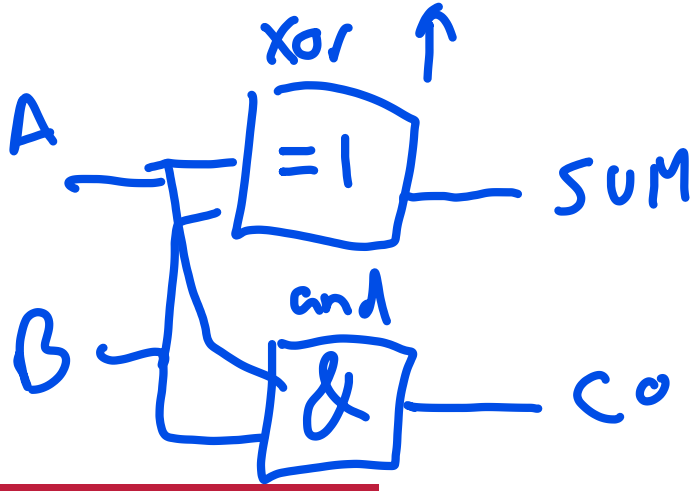
- Bei gleicher Klasse wird die Anweisung von links nach rechts "gelesen"
- Zur besseren Lesbarkeit wird die Nutzung von Klammern empfohlen

```
not X and Y xor Z rol 1 <-> ((not X) and Y) xor (Z rol 1)
```

Modellierung von Halbaddierern

input output

A	B	SUM	CO
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1




```
entity HALBADD is
  port ( A, B : in std_ulogic;
         SUM, CO : out std_ulogic);
end HALBADD;
architecture BEH of HALBADD is
begin
  SUM <= A xor B;
  CO <= A and B;
end BEH;
```

Modellierung von Halbaddierern

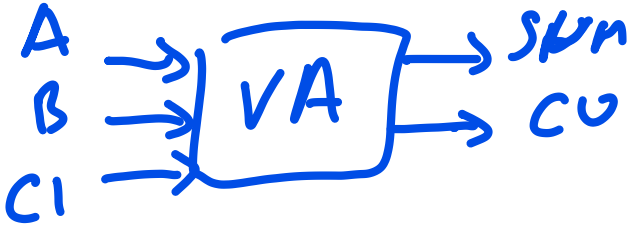
A	B	SUM	CO
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

```
entity HALBADD is
  port (A, B    : in  std_ulogic;
        SUM, CO : out std_ulogic);
end HALBADD;

architecture BEHAVIORAL of HALBADD is
begin
  SUM <= A xor B;
  CO  <= A and B;
end BEHAVIORAL;
```

Halbaddierer

Modellierung von Volladdierern



CI	A	B	SUM	CO	
0	0	0	0	0	0
0	0	1	1	0	1
0	1	0	1	0	1
0	1	1	0	1	2
1	0	0	1	0	1
1	0	1	0	1	2
1	1	0	0	1	2
1	1	1	1	1	3

Modellierung von Volladdierern

CI	A	B	SUM	CO
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

```
entity VOLLADD is
  port (A, B, CI : in  std_ulogic;
        SUM, CO  : out std_ulogic);
end VOLLADD;

architecture BEHAVIORAL of VOLLADD is
 signal CG1, CG2, CP, SUM_INT : std_ulogic;
 component HALBADD
   port (A, B    : in  std_ulogic;
         SUM, CO : out std_ulogic);
begin
HA1: HALBADD port map (A, B, CP, CG1);
HA2: HALBADD port map (CP, CI, SUM_INT, CG2);
CO  <= CG1 or CG2;
SUM <= SUM_INT;
end BEHAVIORAL;
```

Volladdier

Nebenläufige Signalzuweisungen

In VHDL gibt es drei Arten von nebenläufigen Signalzuweisungen:

- Die **unbedingte Signalzuweisung**. Die Zuweisung erfolgt unabhängig vom Zustand anderer Signale.

```
a <= b;
b(7 downto 4) <= a(7 downto 4);
c <= (others => '1');
```

- Die **selektive Signalzuweisung**, die funktional gesehen einer Auswahl aus einer Reihe von gleichberechtigen Möglichkeiten entspricht, die in Digitaltechnik als Multiplexer bezeichnet wird.
- Die **bedingte Signalzuweisung**, bei der die Zuweisung des aktuellen Signalwerts abhängig von der Auswertung einer oder mehrerer Bedingungen ist. In der Digitaltechnik wird dieses Verhalten durch einen Prioritätsencoder realisiert.

Selektive Signalzuweisungen

```
[<Bezeichner> :]
with <Signal-Kombination> select
  <Signalname> <= <logische Ausdruck 1> when <Sig_Wert_1> ,
                  [<logische Ausdruck 2> when <Sig_Wert_2> , ]
                  ...
                  [<Logische Ausdruck n> when others ];
```

- Der Bezeichner ist optional. Er dient in den Simulations- bzw. Synthesewerkzeugen zur Kennzeichnung von hierarchischen Strukturen bzw. Signalen.
- Die "`when others`" Verzweigung ist immer dann zwingend erforderlich, wenn die Anzahl der zuvor angegebenen Kombinationen der Signalwerte `Sig_Wert_i` kleiner ist, als die Anzahl der prinzipiell möglichen Kombinationen.

Bedingte Signalzuweisungen

```
[<Bezeichner> :]
  <Signalname> <= <logische Ausdruck 1> when <Bedingung_1> else
                  [<logische Ausdruck 2> when <Bedingung_2> else ,]
                  ...
                  <Logische Ausdruck n>;
```

- Die bedingte Signalzuweisung (conditional signal assignment) entspricht in ihrer Funktion `if-then-else` einer Strukturauswahl, wie sie aus den meisten Programmiersprachen bekannt ist.
- In der synthetisierten Hardware führt eine bedingte Signalzuweisung mit mehreren Bedingungsausdrücken hingegen zu einer Schachtelung von Gatterstufen, die einer Prioritätsencoderstruktur entspricht.
- Im Unterschied zur selektiven Signalzuweisung können prinzipiell auch völlig unterschiedliche Signale bzw. deren Kombinationen in den einzelnen Bedingungen verwendet werden.

Nebenläufige Signalzuweisungen: Beispiel

```
entity MUX2ZU1 is
  port (I : in std_ulogic_vector(1 downto 0);
        S, E : in std_ulogic; -- Select und Enable Eingaenge
        Y : out std_ulogic);
end MUX2ZU1;

architecture MUX1 of MUX2ZU1 is
begin
  Y <= (I(0) and not E and not S) or (I(1) and not E and S);
end MUX1;
```

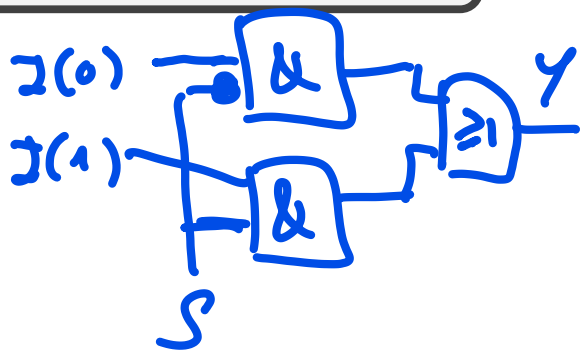
Enable

I(0) I(1)

E	S	Y
0	0	I(0) or 0 = I(0)
0	1	0 or I(1) = I(1)
1	0	0
1	1	0

enable true

0



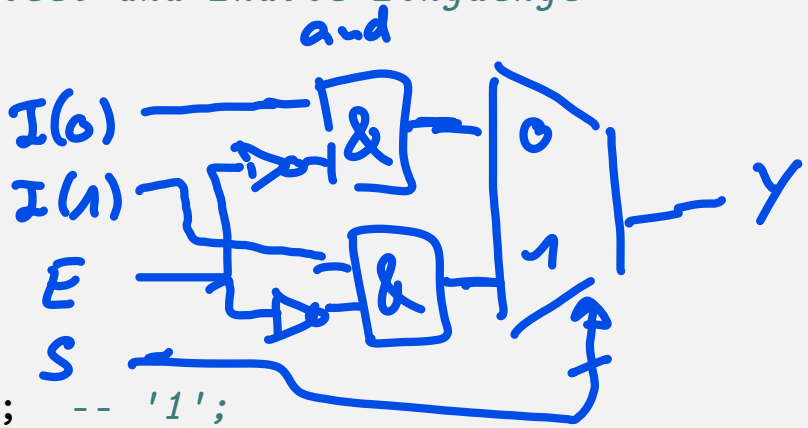
Nebenläufige Signalzuweisungen: Beispiel

PRIORITY DECODER MULTIPLEXER

```
entity MUX2ZU1 is
  port (I : in std_ulogic_vector(1 downto 0);
        S, E : in std_ulogic; -- Select und Enable Eingaenge
        Y : out std_ulogic);
end MUX2ZU1;

architecture MUX2 of MUX2ZU1 is
begin
  with S select
  Y <= (I(0) and not E) when '0',
       (I(1) and not E) when others;  -- '1';
end MUX2;

architecture MUX3 of MUX2ZU1 is
begin
  Y <= (I(0) and not E) when S='0' else
       (I(1) and not E);
end MUX2;
```



Nebenläufige Signalzuweisungen

Literatur

- Jürgen Reichardt, Bernd Schwarz, "VHDL-Simulation und -Synthese", Walter de Gruyter GmbH, 2020
- Janick Bergeron, "Writing Testbenches: Functional Verification of HDL Models", Springer Science & Business Media, 2012.
- Peter J. Ashenden, "The Designer's Guide to VHDL", Morgan Kaufmann, 2010.